

DBMS, OS & DSA Interview Prep — MCQs & Implementations

Covers indexing, caching, deadlock detection, graph algorithms, and tries. Correct answer marked with . Short justification given after each.

1. Why are B-Trees and B+ Trees preferred for database indexing?

A) They store data in a hash map for $O(1)$ access B) They keep data sorted and minimize disk I/O by keeping the tree height small  C) They only work well for in-memory data D) They eliminate the need for indexing altogether

Why: Disk access is the bottleneck in DBMS. B-Trees/B+ Trees are wide, shallow (high fan-out) structures, so even huge tables need only 3-4 disk reads to locate a record, and they preserve sorted order for range scans.

2. What is the difference between B-Tree and B+ Tree?

A) B-Tree stores data only in leaf nodes; B+ Tree stores data in all nodes B) B+ Tree stores data only in leaf nodes (linked together); B-Tree stores data in internal nodes too  C) B-Tree does not support search; B+ Tree does D) There is no real difference, they are the same structure

Why: In a B+ Tree, internal nodes hold only keys (for navigation), and all actual records sit in leaf nodes that are linked in a list — this makes range queries and full scans fast. B-Trees store data in internal nodes too, so range queries require in-order traversal with backtracking.

3. How do LRU and LFU caches work internally?

A) LRU evicts the least recently used item; LFU evicts the least frequently used item  B) LRU evicts the most recently used item; LFU evicts the most frequently used item C) Both evict items randomly when full D) LRU and LFU are identical algorithms with different names

Why: LRU tracks recency (usually via a doubly linked list + hashmap). LFU tracks usage count (a frequency counter alongside the hashmap), evicting the item used least often, breaking ties by recency.

4. Implement LRU Cache and explain time complexity

A) Using only an array — $O(n)$ get and $O(n)$ put B) Using a HashMap + Doubly Linked List

— $O(1)$ get and $O(1)$ put  C) Using a sorted TreeMap — $O(\log n)$ get and $O(\log n)$ put D)
Using a Stack — $O(1)$ get but $O(n)$ put

Why: HashMap gives $O(1)$ lookup of a node; the doubly linked list lets you move that node to the front (most recently used) or evict from the back (least recently used) in $O(1)$, since you have direct pointers — no traversal needed.

Java Implementation

```
java
```

```
import java.util.HashMap;

class LRUCache {
    private class Node {
        int key, value;
        Node prev, next;
        Node(int key, int value) {
            this.key = key;
            this.value = value;
        }
    }

    private final int capacity;
    private final HashMap<Integer, Node> map;
    private final Node head, tail; // dummy sentinels

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.map = new HashMap<>();
        head = new Node(-1, -1);
        tail = new Node(-1, -1);
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key);
        remove(node);
        insertAtFront(node);
        return node.value;
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            remove(map.get(key));
        }
        if (map.size() == capacity) {
            map.remove(tail.prev.key);
            remove(tail.prev);
        }
        Node node = new Node(key, value);
        map.put(key, node);
        insertAtFront(node);
    }
}
```

```

private void remove(Node node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
}

private void insertAtFront(Node node) {
    node.next = head.next;
    node.prev = head;
    head.next.prev = node;
    head.next = node;
}
}

```

Time complexity: `get()` $\rightarrow O(1)$, `put()` $\rightarrow O(1)$. **Space complexity:** $O(\text{capacity})$.

5. Implement LFU Cache and explain why it is harder than LRU

A) LFU is easier than LRU because it only needs a HashMap B) LFU is harder because it must track both frequency and recency (tie-breaking), requiring a frequency-bucketed structure  C) LFU is harder because it cannot be implemented in $O(1)$ time under any circumstances D) LFU and LRU have identical implementation complexity

Why: LRU needs only one dimension of ordering (recency). LFU needs two: frequency count, and — when multiple items share the same frequency — recency among them. This requires a map of frequency $\rightarrow$ doubly linked list of nodes at that frequency, plus tracking the minimum frequency currently in use, to keep both `get()` and `put()` at $O(1)$.

Java Implementation ($O(1)$ LFU)

```

java

```

```
import java.util.HashMap;

class LFUCache {
    private class Node {
        int key, value, freq = 1;
        Node prev, next;
        Node(int key, int value) {
            this.key = key;
            this.value = value;
        }
    }

    private class DoublyLinkedList {
        Node head = new Node(-1, -1), tail = new Node(-1, -1);
        int size = 0;
        DoublyLinkedList() {
            head.next = tail;
            tail.prev = head;
        }
        void addFront(Node node) {
            node.next = head.next;
            node.prev = head;
            head.next.prev = node;
            head.next = node;
            size++;
        }
        void remove(Node node) {
            node.prev.next = node.next;
            node.next.prev = node.prev;
            size--;
        }
        Node removeLast() {
            if (size == 0) return null;
            Node last = tail.prev;
            remove(last);
            return last;
        }
    }

    private final int capacity;
    private int minFreq;
    private final HashMap<Integer, Node> keyMap;
    private final HashMap<Integer, DoublyLinkedList> freqMap;

    public LFUCache(int capacity) {
        this.capacity = capacity;
    }
}
```

```

        this.minFreq = 0;
        this.keyMap = new HashMap<>();
        this.freqMap = new HashMap<>();
    }

    public int get(int key) {
        if (!keyMap.containsKey(key)) return -1;
        Node node = keyMap.get(key);
        updateFreq(node);
        return node.value;
    }

    public void put(int key, int value) {
        if (capacity <= 0) return;

        if (keyMap.containsKey(key)) {
            Node node = keyMap.get(key);
            node.value = value;
            updateFreq(node);
            return;
        }

        if (keyMap.size() == capacity) {
            DoublyLinkedList list = freqMap.get(minFreq);
            Node evicted = list.removeLast();
            keyMap.remove(evicted.key);
        }

        Node node = new Node(key, value);
        keyMap.put(key, node);
        freqMap.computeIfAbsent(1, k -> new DoublyLinkedList()).addFront(node);
        minFreq = 1;
    }

    private void updateFreq(Node node) {
        int oldFreq = node.freq;
        freqMap.get(oldFreq).remove(node);
        if (oldFreq == minFreq && freqMap.get(oldFreq).size == 0) {
            minFreq++;
        }
        node.freq++;
        freqMap.computeIfAbsent(node.freq, k -> new DoublyLinkedList()).addFront
    }
}

```

Time complexity: $\text{get}() \rightarrow O(1)$, $\text{put}() \rightarrow O(1)$. **Space complexity:** $O(\text{capacity})$.

6. How is page replacement related to cache eviction policies?

- A) They are unrelated concepts from different fields B) Page replacement (OS) and cache eviction (systems/DBMS) both solve the same core problem — deciding what to remove when a fixed-size store is full — using the same algorithms like LRU, LFU, FIFO, and Clock ☒ C) Page replacement is only about disk formatting D) Cache eviction only applies to CPU registers

Why: Both are instances of the general "which item to discard under a capacity constraint" problem. OS page replacement (deciding which memory page to swap out) and application-level cache eviction (deciding which entry to remove) use identical strategies: LRU, LFU, FIFO, Optimal (Belady's), and Clock/Second-Chance approximations.

7. How does a hash index differ from a tree index in DBMS?

- A) Hash index supports range queries better than tree index B) Hash index gives $O(1)$ average lookup for equality but cannot support range queries; tree index (B+ Tree) supports both equality and range queries in $O(\log n)$ ☒ C) Tree index is always faster than hash index for every query type D) Hash indexes maintain sorted order of keys

Why: A hash index maps keys to buckets via a hash function — great for `WHERE x = value` but useless for `WHERE x BETWEEN a AND b` since hashing destroys ordering. A B+ Tree index preserves sorted order, so both equality and range predicates are efficient.

8. Why is a B+ Tree better for range queries than a hash table?

- A) B+ Tree leaf nodes are linked in sorted order, so a range query just walks the leaf chain after one lookup; a hash table has no ordering, forcing a full scan ☒ B) Hash tables are always slower than B+ Trees for every operation C) B+ Trees don't require any traversal for range queries D) Hash tables store data in sorted buckets by default

Why: Once you locate the starting key in a B+ Tree ($O(\log n)$), you can sequentially read forward along the linked leaf nodes to get the full range in $O(k)$ for k results. A hash table would need to scan every bucket since there's no relationship between key value and bucket location.

9. How can deadlock be detected using a graph?

- A) By checking if the graph is fully connected B) By building a Wait-For Graph (WFG) where an edge $P_i \rightarrow P_j$ means process P_i is waiting on a resource held by P_j , and checking for

a cycle ☒ C) By counting the number of nodes in the graph D) By checking if the graph is a tree

Why: Each process is a node; a directed edge from process A to process B means A is blocked waiting for a resource B holds. If this graph contains a cycle, those processes are stuck waiting on each other indefinitely — a deadlock.

10. What is the relation between wait-for graph cycle detection and deadlock?

A) A cycle in the wait-for graph is a necessary and sufficient condition for deadlock (in a single-instance-per-resource-type system) ☒ B) A cycle in the wait-for graph only indicates high CPU usage C) Deadlock can occur only if the wait-for graph has no edges D) Wait-for graphs cannot detect deadlocks, only prevent them

Why: When each resource type has exactly one instance, a cycle in the WFG means every process in that cycle is waiting for something held by the next process in the cycle — none can proceed, which is the definition of deadlock. (For resources with multiple instances, cycle presence is necessary but not always sufficient; you also need resource-allocation graph reduction.)

11. Why is topological sorting useful in scheduling and dependency resolution?

A) It randomly orders independent tasks B) It produces a linear ordering of tasks such that every dependency (edge $u \rightarrow v$) is satisfied before the dependent task runs — only possible on a Directed Acyclic Graph (DAG) ☒ C) It only works on graphs with cycles D) It sorts tasks alphabetically regardless of dependencies

Why: Build tools, package managers, and task schedulers model "must run before" relationships as directed edges. Topological sort gives a valid execution order (e.g., compile module A before module B if B depends on A). If a cycle exists, topological sort is impossible — which is exactly how these systems detect circular dependencies.

12. How does shortest path relate to routing in computer networks?

A) Routing protocols are unrelated to shortest path algorithms B) Routers model the network as a graph (nodes = routers, edges = links with weights like latency/cost) and use shortest-path algorithms (Dijkstra's for link-state, Bellman-Ford for distance-vector) to find the optimal path for packets ☒ C) Routing always uses a fixed static path regardless of network conditions D) Shortest path algorithms are only used for GPS navigation, not networking

Why: Protocols like OSPF use Dijkstra's algorithm (link-state routing) and protocols like RIP use a Bellman-Ford variant (distance-vector routing) to compute the lowest-cost path from each router to every destination, updating as link weights (congestion, latency, failures) change.

13. Why does DNS lookup, caching, and routing benefit from data-structure design?

A) DNS and routing don't rely on data structures — they use random guessing B) DNS lookup uses hierarchical tree-like namespace resolution, caching uses hashmaps/LRU for fast repeated lookups, and routing uses tries (for prefix-based IP matching) and graphs (for path computation) — good data structures directly reduce latency and lookup cost ☒ C) All of DNS, caching, and routing use only linked lists internally D) Data structures are irrelevant once data is small enough to fit in RAM

Why: DNS resolution is inherently hierarchical (root → TLD → authoritative), naturally suited to a tree. DNS resolvers cache records using hashmap + TTL-based eviction (similar to LRU) to avoid repeated network round trips. Routers use a Patricia trie / compressed trie for longest-prefix-match on IP addresses, since it's far faster than linear scanning routing tables.

14. How do tries support prefix-based search and autocomplete?

A) A trie stores characters along paths from root to leaf, so all words sharing a prefix share the same path — this lets you find all completions of a prefix in $O(\text{prefix length} + \text{number of matches})$ rather than scanning every word ☒ B) A trie stores entire words as single hash values, making prefix search impossible C) Tries are only used for sorting numbers D) Tries require scanning the whole dataset for every prefix query

Why: Each node in a trie represents a character; walking down capacity characters of a prefix takes you directly to the subtree containing every word with that prefix. From there, a DFS collects all completions — no need to compare against unrelated words, unlike a flat list or hashmap.

15. What is the tradeoff between indexing speed and update cost in DBMS?

A) More indexes always make both reads and writes faster B) Indexes speed up reads (SELECT/lookups/range queries) but slow down writes (INSERT/UPDATE/DELETE), since every index must also be updated whenever the underlying data changes ☒ C) Indexes have no effect on write performance D) Indexes only affect storage size, never speed

Why: Every index is an additional data structure (usually a B+ Tree) that must stay in sync with the table. So an INSERT doesn't just write one row — it also updates every index on

that table, which costs extra disk I/O and CPU. This is the classic read/write tradeoff behind index design: index the columns you query often, not every column.

Quick Reference Summary

Topic	Key Insight
B-Tree vs B+ Tree	B+ Tree keeps data only in linked leaves → better range queries
LRU	HashMap + Doubly Linked List → $O(1)$ get/put
LFU	HashMap of key→node + HashMap of freq→DLL + minFreq tracker → $O(1)$ get/put
Page replacement	Same algorithms (LRU/LFU/FIFO/Clock) as cache eviction
Hash index	$O(1)$ equality, no range query support
Deadlock detection	Cycle in Wait-For Graph
Topological sort	Valid task order on a DAG; fails on cycles
Shortest path	Dijkstra (link-state) / Bellman-Ford (distance-vector) for routing
Tries	Shared prefixes → fast autocomplete
Indexing tradeoff	Faster reads, slower writes